

Главный Claude нанимает отдел стажёров.

До этого Claude был один. Сегодня учим его нанимать команду стажёров: subagents, которые работают параллельно, каждый в своём изолированном контексте. Делаем *due diligence* на акцию через четыре параллельных аналитика и собираем *code review*-команду из трёх специалистов.

ДЛИТЕЛЬНОСТЬ

8–10 дней · 2–4 ч/день

ЧТО ВЫ ОСВОИТЕ

subagents · параллелизм · brief · worktree-isolation · multi-agent teams

ГЛАВНЫЙ ИТОГ

задачи на «4 человека» решаются за один запуск

АРТЕФАКТ

инвестиционный мемо по акции и тройной code review проекта 06

Subagents встроены в Claude Code, ставить их не нужно. Но два skill из набора `superpowers` делают работу с ними дисциплинированной. Если вы ставили `superpowers` в этапах 04 или 05 — всё уже на месте.

ОБЯЗАТЕЛЬНО

Из этапов 00–06

- Claude Code, MCP `@playwright/mcp`, `@upstash/context7-mcp`, skill `frontend-design`;
- проект `portfolio-journal` или личный инвест-кабинет (используем как «реальные данные»);
- git и GitHub из этапа 05 — для `worktree-isolation`.

ЖЕЛАТЕЛЬНО • SKILL

`superpowers:dispatching-parallel-agents`

Что это: skill, который учит Claude правильно *отправлять* несколько subagents параллельно: формулировать чёткие brief, ждать всех, обрабатывать ошибки одного, не валя весь конвейер.

Зачем сейчас: для Due Diligence нужно запустить четырёх аналитиков одновременно. Без skill Claude может запустить их по очереди — и время ответа вырастет в четыре раза.

Проверить: `/skills` — skill в списке.

```
# внутри claude (slash-команда, не shell). Пакет тот же, что в этапе 04
/plugin install superpowers@claude-plugins-official
```

ЖЕЛАТЕЛЬНО • SKILL

`superpowers:subagent-driven-development`

Что это: skill про *стиль* работы с subagents: когда делегировать, какой brief давать, как читать результат, как избегать «уплываний».

Зачем сейчас: второй проект — code review-команда — работает гораздо лучше с этим skill.

Где взять: входит в тот же пакет `superpowers`.

ЖЕЛАТЕЛЬНО • SKILL

`superpowers:using-git-worktrees`

Что это: skill для дисциплины worktrees из этапа 05.

Зачем сейчас: «грязные» subagents (которые вносят изменения в файлы) лучше изолировать в worktree, чтобы они не мешали вашей основной работе. Особенно когда запускаются 3–

4 параллельно.

Что такое subagent: «стажёр на одну задачу»

Subagent — это *отдельный экземпляр Claude*, которого вы запускаете на одну подзадачу. У него:

- **Свой контекст.** Он *не видит* вашей переписки с главным Claude. Получает только тот brief, который вы ему дали.
- **Свой набор инструментов.** Можно дать ему доступ к файлам, можно ограничить только web-search — зависит от типа.
- **Один результат.** Он возвращает *одно сообщение* с итогом своей работы. Не ведёт диалог — делает задачу и уходит.

Метафора, которую полезно держать в голове весь этап: вы нанимаете *стажёра* для одной задачи. Стажёр умный, но не знает, чем занят остальной офис. Вы даёте ему **brief**: «вот данные, вот задача, вот формат ответа», ждёте, получаете отчёт.

Главные плюсы:

- **Параллелизм.** Можно запустить 3–4 стажёра одновременно. Что вручную заняло бы 4 часа, делается за один.
- **Изоляция контекста.** Вы экономите свой основной контекст: «грязная» работа делается в контексте стажёра, вам приходит только итог.
- **Специализация.** Можно нанять «code-reviewer» с его настроенным промптом и поведением, не объясняя каждый раз, что вы хотите от ревью.

Типы subagents в Claude Code

Claude Code приносит несколько готовых типов. Названия могут отличаться по вашей конфигурации, но логика общая:

- **general-purpose** — универсальный стажёр без специализации. Можно дать любую задачу.
- **Explore / code-explorer** — «следователь по коду»: пройди по файлам, найди где определена функция X, что в этом проекте необычного. Только чтение, без правок.
- **code-reviewer** — ревью кода: ищет баги, проблемы безопасности, отклонения от конвенций.

- **code-architect** – архитектура: проектирует решение, прежде чем писать код.
- **code-simplifier** – упрощает код: чище, короче, читаемее.
- **plan-agent** – «планировщик»: разбивает крупную задачу на шаги.
- и другие, которые могут появиться от установленных плагинов.

Вызов – через инструмент `Task` (главный Claude использует его сам, вы просто говорите):

```
> используй subagent code-reviewer на @backend/auth.py.
> задача: найди риски безопасности и предложи исправления.
> формат: список из 3-5 пунктов, по убыванию серьёзности.
```

Список доступных – через `/agents` (или эквивалент в вашей конфигурации).

Brief: как нанять стажёра правильно

Brief – это технический «вход» для subagent. От его качества напрямую зависит, насколько полезен будет результат. Хороший brief содержит:

1. **Цель в одной фразе.** «Найди три риска безопасности в auth.py».
2. **Входы.** Какие файлы читать, какие данные использовать. Лучше через `@`-ссылки.
3. **Чего НЕ делать.** «Не трогай файлы, только читай». «Не используй сторонние сервисы». «Не давай советов по UX, только по безопасности».
4. **Формат ответа.** «Список из 3–5 пунктов в Markdown». «JSON со структурой X».
5. **Объём.** «Одна страница максимум». «По 5–7 строк на каждый риск».

Пример хорошего brief для инвест-аналитика:

```
> запусти subagent general-purpose со следующим brief:
>
> РОЛЬ: фундаментальный аналитик акций.
> ЦЕЛЬ: оцени фундаменталы AAPL за последний квартал.
> ВХОДЫ: можешь использовать WebSearch и WebFetch для
>   сайта investor.apple.com и финансовых лент.
> НЕ ДЕЛАЙ: не давай прогнозов. не используй технический анализ.
>   не пиши «по моему мнению» – только факты.
> ФОРМАТ: markdown-блок с разделами:
>   - выручка (yoу +/-);
>   - чистая прибыль (yoу +/-);
>   - P/E vs средний по сектору;
>   - три ключевых факта из последнего квартала.
> ОБЪЁМ: одна страница максимум.
```

Принцип «fail fast в brief»: лучше потратить 30 секунд на точный brief, чем 5 минут на разбор результата, в котором стажёр сделал «не то».

Параллельный запуск: несколько стажёров одновременно

Главный плюс subagents – параллелизм. Если задачи независимы (один анализирует фундаменталы, второй – график, третий – новости), запускаем их *одновременно*.

В Claude Code для этого – параметр `run_in_background: true`. Один промпт – четыре стажёра в работе:

```
> для AAPL запусти ПАРАЛЛЕЛЬНО (run_in_background) четырёх subagents:
>
> 1) ФУНДАМЕНТАЛЬНЫЙ – выручка, прибыль, P/E (brief 1).
> 2) ТЕХНИЧЕСКИЙ – тренд, индикаторы, поддержки (brief 2).
> 3) НОВОСТНОЙ – sentiment по 30 свежим заголовкам (brief 3).
> 4) МАКРО – отрасль и экономический фон (brief 4).
>
> ЖДИ всех. КАЖДЫЙ возвращает свой markdown-блок.
> ПОТОМ собери в один файл мемо-aapl.md с разделами по каждому.
```

Что произойдёт:

- Главный Claude отправит четыре `Task`-вызова, каждый с `run_in_background: true`;
- Они работают параллельно (~2–4 минуты вместо ~10–15);
- Главный Claude дожидается всех, читает результаты;
- Сшивает в единый отчёт.

Когда параллелизм *не нужен*: если результат одного стажёра – вход для другого. Тогда последовательно (foreground): дождались первого, передали второму.

Foreground vs background: когда какое

Foreground – «жду стажёра, прежде чем продолжить». Главный Claude посылает задачу, останавливается, дожидается результата.

- Используется, когда результат подзадачи нужен *прямо сейчас* для следующего шага;
- Удобно для «одного стажёра на разовую задачу»;
- Плюс: проще читать – результат сразу в диалоге.

Background – «отправил задачу, продолжаю работать, заберу результат позже».

- Используется для *параллельных независимых задач* (как четыре аналитика);
- Удобно, когда задача длинная (например, анализ всех файлов проекта);
- Плюс: общее время работы сокращается в разы.

Простое правило:

- Один стажёр + результат сейчас → foreground.
- 3+ стажёра одновременно → background.
- Длинная задача (5+ минут), хочется параллельно делать что-то ещё → background, через какое-то время «проверь, готово ли».

Worktree-isolation: «грязные» стажёры в своей песочнице

Если subagent *читает* – его можно запускать «как есть»: он ничего не ломает. Но если он *пишет* файлы (например, рефакторит код), вы не хотите, чтобы три параллельных стажёра одновременно правили один и тот же `app.py`.

Решение – **worktree-isolation**: каждый «грязный» стажёр работает в собственной копии репозитория (worktree, см. этап 05). После завершения вы смотрите на каждый результат отдельно, выбираете, что мержить, что отбрасывать.

```
> для рефакторинга frontend/auth.tsx запусти параллельно:
> – code-simplifier в worktree feature/auth-simplify;
> – code-reviewer в worktree review/auth (только чтение, ему worktree не об
> – code-architect в worktree feature/auth-redesign.
>
> ЖДИ всех. покажи мне три предложенных версии,
> чтобы я выбрал, что мержить в main.
```

Skill `superpowers:using-git-worktrees` добавляет это автоматически: «грязные» subagents всегда получают свой worktree, чистые – нет.

Multi-agent teams: координация и передача результатов

Простой шаблон – «один главный, четыре параллельных» – работает для independent задач. Но есть и более сложные сценарии:

1. **Конвейер**. Агент А собирает данные → агент В анализирует → агент С пишет отчёт. Каждый – в foreground, выход одного передаётся как вход следующему.

2. **Конкурс.** Три агента решают одну задачу *разными способами*, главный выбирает лучший. Полезно, когда подход неочевиден.
3. **Дискуссия.** Агент А пишет план, агент В критикует, агент А переписывает с учётом критики. Полезно для сложных архитектурных решений.

Главный принцип: *главный Claude – всегда один*. Он дирижёр оркестра, а subagents – музыканты. Стажёр стажёру не звонит – всё координирует главный.

Когда задача требует именно «дирижёра», полезен skill `superpowers:dispatching-parallel-agents`: он учит главного Claude правильно строить такие схемы и собирать результаты.

Когда subagents *не* нужны

Subagents – не «серебряная пуля». Они платят накладные расходы (запуск контекста, передача brief), и иногда проще без них.

Не используйте subagents:

- Для мелких правок, на которые главный Claude отвечает за 30 секунд.
- Когда задачи зависят друг от друга в нелинейном порядке (постоянная необходимость «в ответ на X сделать Y» – на это нужен диалог, а не одноразовый brief).
- Для интерактивных сценариев («давай попробуем», «нет, не так»). Subagent делает *один* ответ – вы можете запросить ещё, но это уже новый стажёр со своим контекстом.

Используйте, когда:

- Задача независима и «крупная» (минут на 5–15);
 - Нужны 2+ независимых результата;
 - Вы не хотите засорять основной контекст «грязной» работой (логи, поиск по коду, парсинг страниц);
 - Есть специализированный тип, идеально подходящий (code-reviewer для ревью, и т. д.).
-

PRACTICUM

Прогон · смоук-тесты subagents

1. Зайдите в ваш проект `portfolio-journal`. Запустите `claude`.

2. **Один subagent (foreground)**. Промпт:

```
> запусти subagent code-explorer на @data/operations.csv.  
> brief: опиши, какие тикеры в файле, сколько уникальных,  
> какой период покрывает, какие колонки – в формате 4-5 строк.  
> ЖДИ результата.
```

3. Когда вернётся – обратите внимание: вы видите *его* ответ, но не его «мысли». Это нормально.

4. **Два параллельных (background)**. Промпт:

```
> ПАРАЛЛЕЛЬНО (run_in_background) запусти двух subagents:  
>  
> 1) general-purpose:  
>   brief: оцени ликвидность моего портфеля  
>   (доля каждого тикера в общем капитале).  
>   источник: @data/operations.csv.  
>   формат: markdown-таблица.  
>  
> 2) general-purpose:  
>   brief: посчитай реализованный P&L по FIFO,  
>   верни топ-5 удачных и неудачных сделок.  
>   источник: @data/operations.csv.  
>   формат: markdown.  
>  
> ЖДИ ОБОИХ. собери в один файл analysis.md.
```

5. Заметьте: время ожидания одно ($\approx$ время самого медленного), а не сумма двух. Это и есть выгода параллелизма.

6. **Четыре параллельных (background)**. Подготовка к первому проекту:

```
> для тикера AAPL ПАРАЛЛЕЛЬНО запусти четыре general-purpose subagents:  
>  
> 1) ФУНДАМЕНТАЛЬНЫЙ – выручка / прибыль / P/E (через WebFetch);  
> 2) ТЕХНИЧЕСКИЙ – текущий тренд, поддержки;  
> 3) НОВОСТНОЙ – sentiment 20 свежих заголовков;  
> 4) МАКРО – отрасль и общий фон рынка.  
>  
> всем – формат markdown, 1 страница, БЕЗ прогнозов и советов.  
> ЖДИ ВСЕХ. покажи мне их сырые ответы по очереди,  
> не сшивая в отчёт – просто по одному.
```

7. Прочитайте каждый ответ отдельно. Заметьте: каждый стажёр *не знает*, что есть остальные.

У каждого свой brief, свой контекст.

8. Финальная команда: «теперь сшей четыре блока в один *teto-aapl.md*, в конце добавь свой собственный *summary* в три предложения как «дирижёр оркестра»». Эту работу делает главный Claude, не subagent.

Что мы тренируем: **лестницу 1→2→4 параллельных стажёров**, чтение их независимых ответов и роль главного как дирижёра.

ПРОЕКТ А · ПО ИНСТРУКЦИИ

Due Diligence на акцию через четырёх параллельных аналитиков

Цель: для одной акции (возьмём NVDA как пример) собрать инвестиционный мемо: фундаментальный анализ + технический + новостной + макро. Каждый делает отдельный *subagent* параллельно. Главный собирает в единый отчёт `memo-nvda.md` с вердиктом «buy / hold / avoid».

Шаги

1. Создайте папку `dd-nvda`, запустите `claude`. Подготовьте `CLAUDE.md`: «инвестиционные мемо — нейтральный тон, факты, без «я считаю», итоговый вердикт обоснован».
2. Войдите в Plan Mode и попросите Claude составить spec для четырёх briefs. Убедитесь, что у каждого есть:
 - Роль (фундаментальный / технический / новостной / макро);
 - Цель и три-четыре конкретных вопроса, на которые он должен ответить;
 - Источники данных (какие WebFetch разрешены);
 - Формат ответа (markdown, разделы);
 - Объём (одна страница).
3. Утвердите brief'ы. Дайте команду:

```
> план одобрен. ПАРАЛЛЕЛЬНО запусти четыре subagents с этими brief.  
> используй skill superpowers:dispatching-parallel-agents  
> для правильной организации.
```

4. Дождитесь возврата всех. Прочитайте каждый ответ отдельно. Если кто-то «уплыл» (например, новостной начал давать прогнозы — запрещено в brief) — перезапустите его с уточнённым brief.

5. Финальная сшивка:

```
> собери мемо-nvda.md со структурой:  
> ## Резюме (3 предложения, написанные тобой)  
> ## Фундаментальные показатели (от subagent 1)  
> ## Технический анализ (от subagent 2)  
> ## Новостной фон (от subagent 3)  
> ## Макроконтекст (от subagent 4)  
> ## Вердикт: Buy / Hold / Avoid  
>   - 3 аргумента «за»;  
>   - 3 аргумента «против»;  
>   - финальное решение и три ключевых риска.
```

6. Откройте мемо. Прочитайте как редактор: что убедительно, что нет.

7. Финальный аудит — «red team»:

```
> запусти ещё одного subagent — «red-team аналитик».  
> brief: пройди по мемо-nvda.md, найди 3 слабых места в моём  
> вердикте. с какими аргументами я бы прозвучал убедительнее  
> для скептика-инвестора?  
> формат: 3 пункта по 4-5 строк.
```

8. Добавьте раздел «Critique» в мемо, чтобы зафиксировать слабые места — и при следующем DD не повторять.

Что вы здесь освоили

- писать чёткие brief за один план;
- запускать четырёх параллельных subagents и ждать всех;
- собирать их ответы в связный документ как «дирижёр»;
- добавлять «red team» — пятого subagent для критики собственного результата.

Code Review-команда на ваш проект из этапа 06

Цель: запустить три параллельных специалистов – `code-reviewer`, `security-reviewer` и `code-simplifier` – на `backend` инвест-кабинета. Получить три независимых отчёта и объединить в план улучшений.

Шаги

1. Зайдите в корень `my-investing` (или вашего инвест-кабинета). Запустите `claude`.
2. Промпт на старт:

```
> ПАРАЛЛЕЛЬНО запусти трёх subagents на @backend/:  
>  
> 1) code-reviewer (или general-purpose с ролью code-reviewer):  
>   brief: ревью качества кода. найди 5 главных проблем  
>   (баги, мёртвый код, нарушения конвенций, неточности).  
>   формат: markdown, по убыванию серьёзности.  
>  
> 2) general-purpose с ролью security-reviewer:  
>   brief: ревью безопасности. SQL-инъекции, утечки секретов,  
>   проблемы с авторизацией, CORS, raw passwords в логах.  
>   формат: markdown, по серьёзности (critical / high / medium / low).  
>  
> 3) code-simplifier (или general-purpose с ролью simplifier):  
>   brief: укажи 5 мест, где код можно упростить  
>   (выделить функцию, убрать дубликат, использовать стандартную библиотеку).  
>   формат: до/после, краткое объяснение.  
>  
> ЖДИ всех. покажи отчёты по одному.
```

3. Прочитайте все три. Заметьте: совпадения часто означают «настоящие» проблемы, расхождения – разные точки зрения.

4. Промпт-сшивки:

```
> собери review.md:  
>   ## Критическое (всё, что critical+high security, и баги code-reviewer)  
>   ## Желательное (medium, конвенции, упрощения)  
>   ## К обсуждению (где мнения расходятся)  
>   для каждого пункта добавь файл и строку, где видна проблема.
```

5. Создайте план улучшений:

```
> составь improvement-plan.md в формате спес:  
>   - три фазы по убыванию срочности;  
>   - в каждой фазе – список задач с критериями готовности;  
>   - примерное время на каждую фазу.
```

6. В Plan Mode запросите план реализации первой фазы. Через TodoWrite пройдите её по шагам (этап 04).

7. После реализации – повторный прогон трёх subagents:

```
> запусти тех же троих ещё раз. сравни с предыдущими отчётами.  
> что закрыто? что осталось? что появилось нового?
```

Что вы здесь освоили

- «независимые точки зрения» как метод – три отчёта вместо одного;
- сводить разные мнения в план: что критично, что желательно, что к обсуждению;
- цикл «ревью → план → реализация → ревью» как стандартная дисциплина улучшения проекта.

ПРОЕКТ С · САМОСТОЯТЕЛЬНО

Due Diligence на вашу акцию

Цель: повторить структуру проекта А для любой акции из вашего собственного портфеля. Те же четыре аналитика, те же brief'ы (но вы их пишете без подсказок), та же сшивка, тот же red-team.

Подсказки

- Если у вас нет «своего» портфеля – возьмите тикер из вашего демо-CSV (этапы 02–03).
- Не копируйте brief'ы из проекта А. Напишите свои – так увидите, что Claude в них уточнит, а что нет.
- В `CLAUDE.md` опишите вашу инвестиционную «философию»: горизонт, риск-аппетит, что считаете «зелёным флагом», что «красным». Это меняет тональность отчёта.
- В конце добавьте раздел «связь с портфелем» – как эта акция вписывается в вашу текущую аллокацию (используйте данные из `portfolio-journal/data/operations.csv`).

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

– сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ в корне есть `memo-<ticker>.md` с шестью разделами (резюме, фундаменталы, технический, новости, макро, вердикт);
- ✓ brief'ы четырёх аналитиков сохранены в `briefs/` (хорошо для будущих DD – повторное использование);
- ✓ четыре subagents запускались *параллельно*, не последовательно (заметно по времени – общее время ≈ времени самого медленного);
- ✓ есть раздел «portfolio fit» с конкретными ссылками на ваш портфель;
- ✓ есть раздел «red team critique» с тремя слабыми местами вашего вердикта;
- ✓ в вердикте присутствуют 3 «за», 3 «против», 3 ключевых риска и финальное «buy / hold / avoid».

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти ещё одного subagent с ролью «инвест-консультант 30-летнего стажа». Brief: прочитай мой тето, оцени – убедил бы он тебя, если бы я принёс это на заседание инвест-комитета? Что бы ты спросил автора?»

«AI-хакатон» — команда для идеи продукта

Цель: вариация структуры «один главный + параллельные специалисты», но другие роли. Дайте Claude идею продукта одной строкой – три *subagents* проходят роли «продакт», «дизайнер», «разработчик», к концу диалога у вас рабочий MVP по ссылке.

Подсказки

- Идея продукта – интересная: «приложение для учёта подаренных книг», «сервис, считающий, сколько кофе я выпил за год», «трекер привычек с еженедельным письмом-резюме».
- Три параллельных *subagents*:
 - **Продакт-менеджер**: 3 user stories, ключевые метрики, 1 главная и 3 второстепенных функции;
 - **Дизайнер**: использует skill `frontend-design`, рисует 3 варианта главного экрана как HTML-моки;
 - **Разработчик**: пишет минимальный прототип (один HTML или микро-FastAPI).
- Главный Claude собирает: выбирает один из трёх дизайнерских вариантов, склеивает с прототипом, деплоит на Vercel или GitHub Pages.
- В конце – короткий `retro.md` в стиле «что у команды получилось, что – нет».

РАСКРЫТЬ ПОСЛЕ САМОСТОЯТЕЛЬНОЙ РАБОТЫ

— сначала сделайте, потом проверяйте

ГОТОВО, ЕСЛИ...

- ✓ есть рабочая ссылка на MVP (Vercel или GitHub Pages);
- ✓ в репо сохранены три «дизайнерские» версии (например, `mock-1.html`, `mock-2.html`, `mock-3.html`);
- ✓ отдельным файлом `product-spec.md` – результат «продакт-менеджера»;
- ✓ `retro.md` содержит хотя бы 3 инсайта про то, что вышло хорошо и 3 про то, что нет;
- ✓ три *subagents* запускались параллельно (видно по логу диалога);
- ✓ при показе MVP постороннему – за 30 секунд понятно, о чём он.

СПРОСИТЕ CLAUDE НАПОСЛЕДОК

«Запусти *subagent* с ролью «инвестор фонда»: прочитай *product-spec* и посмотри MVP. Стал бы ты вкладываться в этот продукт? Какие три вопроса задал бы команде, чтобы решить?»

Что добавилось к словарю

Subagent	Отдельный «стажёр» – изолированный экземпляр Claude со своим контекстом, набором инструментов и одной задачей. Возвращает один итоговый ответ. Не видит вашей переписки с главным Claude.
Brief	«ТЗ» для subagent: цель в одной фразе, входы (через @-ссылки), что НЕ делать, формат ответа, объём. Чем чётче brief, тем полезнее результат.
Task tool	Инструмент Claude Code, через который главный Claude запускает subagent. Параметры: тип агента, brief, foreground/background. Вы вызов не пишете руками – пишете на человеческом языке.
Foreground	Запуск subagent «жду до ответа». Главный Claude останавливается, дожидается результата, продолжает. Для одной разовой задачи.
Background	Запуск subagent «работай в фоне». Главный Claude отправляет задачу и может запускать другие. Для параллельных независимых задач – общее время сокращается в разы.
Параллелизм	Запуск нескольких subagents одновременно. Главное преимущество – общее время равно времени самого медленного, а не сумме всех. Подходит, когда задачи независимы.
Worktree-isolation	Запуск subagent в собственной git-worktree-копии репозитория (этап 05). Используется для «грязных» агентов, которые правят файлы. Параллельные правки не конфликтуют.
Multi-agent team	Несколько subagents под управлением одного главного Claude. Шаблоны: параллельная команда (4 аналитика), конвейер (A→B→C), конкурс (три решения, выбираем лучшее), дискуссия (черновик → критика → переработка).
Дирижёр	Метафора для главного Claude в multi-agent сценарии. Subagents – музыканты; они не общаются друг с другом напрямую. Координирует только дирижёр: даёт brief каждому, собирает результаты, склеивает в общую партитуру.
Red team	Дополнительный subagent, которого вы запускаете в конце с брифом «найди слабости в этом результате». Полезно для Due Diligence, ревью архитектуры, любых решений с высокой ценой ошибки.
Blast radius	«Радиус взрыва» – что максимум может пойти не так, если subagent ошибётся. Для чтения – небольшой. Для правок в файлы – больше, поэтому worktree-isolation. Для деплоев – огромный, лучше не давать subagents.

01 Чем **subagent** отличается от «просто другой задачи в том же диалоге»?

Subagent – *отдельный экземпляр* Claude со своим контекстом. Не видит вашей переписки. Получает только brief, делает одну задачу, возвращает один ответ. «Просто другая задача» – всё в том же контексте, всё видит. Subagent – чистая «коробка», изолированная от шума.

02 Что обязательно должно быть в хорошем **brief**?

Пять пунктов: **цель в одной фразе**; **входы** (какие файлы/данные читать, через @); **что НЕ делать** (явные ограничения); **формат ответа** (markdown / JSON / список); **объём** (одна страница / 5 пунктов). Без каждого – стажёр сделает «не то».

03 Когда **foreground**, когда **background**?

Foreground: один стажёр, результат нужен сейчас для следующего шага. Главный Claude ждёт.
Background: 3+ независимых стажёра, или одна длинная задача, рядом с которой можно делать ещё что-то. Главный отправляет всех и ждёт всех вместе – общее время равно времени самого медленного.

04 Зачем **worktree-isolation**, и для каких **subagents** она *не* нужна?

Нужна для «грязных» субагентов, которые *пишут* в файлы (рефакторинг, генерация кода). Если их запускать без isolation, два параллельных могут конфликтовать – один перезапишет правки другого. *Не нужна* для «чистых» – которые только читают (code-reviewer, аналитики, новостные агенты).

05 Кто координирует команду **subagents**?

Главный Claude. Он «дирижёр»: даёт brief каждому, ждёт ответы, собирает в общий результат. Subagents *не общаются* друг с другом напрямую – они «музыканты», каждый смотрит только в свои ноты. Это упрощает координацию: один источник истины.

06 Что такое «red team» и зачем он на Due Diligence?

Дополнительный subagent в конце процесса с brief'ом «найди слабости в этом результате». Помогает заметить «слепые пятна»: где аргументация хрупкая, какие риски недооценили, что бы спросил скептик. На DD особенно ценно, потому что цена ошибочного вердикта – реальные деньги.

07 Когда subagents не нужны?

Для мелких правок, которые главный Claude делает за 30 секунд. Для интерактивных сценариев («давай попробуем», «нет, не так») – subagent даёт *один* ответ, диалога не ведёт. Для задач с нелинейными зависимостями – на них нужна гибкость, а subagent работает по брифу. Накладные расходы запуска не окупаются.

- ☐ Понимаю, что такое subagent: изолированный «стажёр» со своим контекстом.
- ☐ Знаю основные типы (general-purpose, code-reviewer, code-explorer, и другие).
- ☐ Пишу brief из 5 пунктов (цель, входы, что НЕ делать, формат, объём).
- ☐ Запускаю 3+ subagents параллельно через background и собираю результаты.
- ☐ Использую worktree-isolation для «грязных» агентов, которые правят файлы.
- ☐ Понимаю роль главного Claude как «дирижёра» и собираю отчёты в единый документ.
- ☐ Сделал четыре проекта: DD на NVDA, code-review команда, DD на свою акцию, AI-хакатон.

Готовы закрыть этап? Нажатие фиксирует прогресс на главной странице.

«Один *Claude* думает быстро. Четыре параллельных — в четыре раза быстрее. Главное — чтобы каждому был чёткий *brief*».

Этап 07 пройден — вы умеете нанимать команду стажёров. Дальше превращаем Claude Code в *ваши* личный — через свои skills, slash-команды и hooks.

ВНУТРИ ЭТАПА

- § 0 · Подготовка
- § I · Темы
- § III · Проекты
- § V · Словарь
- § VI · Quiz
- § VII · Чек-лист

КУРС

- К оглавлению
- ← Этап 06
- Этап 08a